AutoForm Bot

Complete Idea & Architecture Document

WhatsApp Bot · FastAPI · Playwright · Gemini AI · Supabase

Team INNOVEX — Pragati Engineering College

1. The Product

AutoForm Bot is a WhatsApp bot that automatically fills and submits Google Forms for students using AI. The user forwards one link to the bot. Everything else is invisible and automatic — the form is opened, questions are read, AI generates answers, the form is submitted, and the score is sent back to the user on WhatsApp.

One Line Summary

> ***User forwards a Google Form link → Bot fills it using AI → User gets the score back in 30 seconds.***

---

2. The Problem It Solves

Every day in every college WhatsApp group, students receive messages like:

```
"Fill this attendance form before 5 PM"
"Submit the feedback form today"
"Register for the event using this link"
```

The current manual process requires the student to:

1. Open the form link
2. Read every question carefully
3. Think of answers for each question
4. Manually fill each field
5. Click Submit and check the score

This takes 5 to 10 minutes per form. With 10 forms per week, that is nearly an hour of repetitive, low-value work every single week. AutoForm Bot eliminates this entirely.

---

3. The Solution

One forward message. Thirty seconds. Done.

What the user does

6. See the form link in their WhatsApp group
7. Long press the message → tap Forward → select the bot number
8. Wait 30 seconds
9. Receive the score

The user never opens the form. Never reads a question. Never types a single answer.

---

4. Why Pure Code Instead of n8n

The previous version of this architecture used n8n as a visual workflow engine. That approach has been completely removed. All workflow logic is now handled directly in Python FastAPI code. Here is why this is the better decision:

Aspect	n8n Approach	Pure Python Approach
Control	Limited — visual only	Full — write any logic
Debugging	Visual dashboard	Python logs, full traceback
Playwright support	Cannot run inside n8n	Native, runs directly
Hosting	Two servers needed	One single server
Cost	Extra Railway server	Just one Render server
Complexity	Two systems to manage	One clean codebase
Customization	Node-limited	Unlimited
Speed	Extra HTTP hop	Direct function calls

Conclusion: everything n8n was doing — receiving messages, validating links, checking profiles, sending replies — is now handled in clean Python code inside FastAPI. The result is a simpler, faster, more powerful system.

---

5. Complete System Architecture

Layer 1 — The User Side

The student sees a form link in their WhatsApp group. They forward it to the bot's WhatsApp number. That is the only action required. Everything after that is automatic.

Layer 2 — Meta WhatsApp Cloud API

Meta's official WhatsApp Cloud API receives any message sent to the bot number. It immediately fires an HTTP POST request to the FastAPI webhook endpoint containing the sender's phone number and the full message text. This happens in under 1 second and is free for up to 1000 conversations per month.

Layer 3 — FastAPI Webhook Handler (Replaces n8n)

FastAPI receives the incoming webhook from Meta. A dedicated router handles all the workflow logic that n8n previously handled — but now it is written directly in Python. The complete decision flow coded in Python is:

- Parse the incoming Meta webhook JSON payload
- Extract the sender phone number and message text
- Immediately return HTTP 200 to Meta to prevent retries
- Run message processing in a background task (async)
- Check: does the message contain a URL?
- Check: is the URL a Google Form link (forms.gle or docs.google.com/forms)?
- Query Supabase: does this user have a saved profile?
- If no profile: send registration instructions via Meta API
- If valid form + profile exists: send 'Processing...' reply to user
- Call the form automation engine with the URL and user profile
- Receive the score result from the automation engine
- Send the final score back to the user via Meta API

Layer 4 — Form Automation Engine (Python)

A dedicated Python module handles the entire form automation. It is called directly as a function from the FastAPI handler — no HTTP hops, no extra servers. The sequence is:

10. Receive form URL and user profile data
11. Launch Playwright — invisible Chrome browser, no window, fully background
12. Open the Google Form URL
13. Scrape all questions, their types, and available options from the page DOM
14. For each question, decide the data source using the profile detector module
15. Personal fields are filled directly from the user profile
16. Quiz and knowledge questions are batched and sent to Gemini AI
17. Gemini returns all answers as a clean JSON array
18. Playwright fills every field with the correct answer
19. Playwright clicks the Submit button
20. Wait for the confirmation page to load
21. Read the score from the confirmation page
22. Return the result to the FastAPI handler

Layer 5 — Supabase Database (User Profiles)

Supabase is a free cloud PostgreSQL database. It stores one record per user, identified by their WhatsApp phone number as the primary key.

Column	Type	Example
phone_number	TEXT (Primary Key)	919876543210

Column	Type	Example
name	TEXT	Rohit
roll_number	TEXT	22EG1A0501
college	TEXT	Pragati Engineering College
branch	TEXT	CSE
year	TEXT	2nd Year
email	TEXT	rohit@example.com
forms_filled	INTEGER	14
created_at	DATE	2025-03-13

Layer 6 — Gemini AI (The Answering Engine)

Gemini 1.5 Flash receives all non-personal questions in a single API call and returns answers as a structured JSON array. It also has full vision capability — when a question contains an image, Playwright screenshots that question block and sends the image alongside the text to Gemini Vision.

Layer 7 — Meta Cloud API Message Sender

A dedicated Python module handles all outgoing WhatsApp messages. It uses the httpx library to make async POST requests to the Meta Graph API. It sends three types of messages: the initial 'Processing...' acknowledgement, the final score reply, and registration or error instructions.

---

6. Complete Tech Stack

Component	Tool	Hosted On	Cost
WhatsApp connection	Meta Cloud API	Meta servers	Free (1000 msg/month)
Backend server + webhook	FastAPI + Python	Render	Free tier
Browser automation	Playwright (Python)	Inside Render	Free
AI answering (text + vision)	Gemini 1.5 Flash	Google AI Studio	Free
Database	Supabase (PostgreSQL)	Supabase cloud	Free tier
Async HTTP client	httpx	Inside Render	Free
Environment config	python-dotenv	Inside Render	Free

Total monthly cost: zero rupees. One single server on Render hosts everything.

---

7. FastAPI Endpoints (All Workflow Logic in Code)

Method	Endpoint	Purpose
GET	/webhook	Meta webhook verification — called once during setup
POST	/webhook	Receives all incoming WhatsApp messages from Meta
POST	/user/save	Save new user profile to Supabase
PUT	/user/update	Update existing user profile
GET	/user/{phone}	Check if a user profile exists by phone number
GET	/health	Server health check for Render uptime monitoring

All workflow decisions (link detection, profile check, form routing, reply sending) happen inside the POST /webhook handler — written entirely in Python. No external automation tool needed.

---

8. Project File Structure

```
whatsapp-form-bot/
│
├── main.py                    ← FastAPI app, all routes defined here
├── requirements.txt           ← All Python dependencies
├── .env                       ← API keys (never commit this file)
│
├── src/
│   ├── webhook_handler.py     ← All workflow logic (replaces n8n)
│   ├── form_bot.py            ← Playwright form automation engine
│   ├── ai_helper.py           ← Gemini AI text + vision integration
│   ├── profile_detector.py    ← Personal field keyword matching
│   ├── database.py            ← Supabase read/write operations
│   └── whatsapp.py            ← Meta API outgoing message sender
│
└── tests/
    └── test_form_bot.py       ← Unit tests for form automation
```

What Each File Does

File	Responsibility
main.py	FastAPI app entry point. Defines /webhook GET and POST routes. Calls webhook_handler.
webhook_handler.py	The brain. Parses Meta payload, validates links, checks profiles, triggers automation, sends replies. This is what n8n was doing — now pure Python.
form_bot.py	Launches Playwright, opens the form, scrapes questions, fills answers, submits, returns score.
ai_helper.py	Sends questions to Gemini API. Handles both text-only and vision (image) requests. Parses JSON response.
profile_detector.py	Takes a question text and checks it against keyword lists. Returns whether to use profile data or send to Gemini.
database.py	All Supabase operations — fetch user by phone, save new user, update existing user, increment forms_filled counter.
whatsapp.py	Sends WhatsApp messages via Meta Graph API using httpx. Handles text replies and error messages.

---

9. User Profile System

Registration Flow

The first time a user sends a form link, webhook_handler.py queries Supabase and finds no profile. It sends registration instructions. Once the user replies with their details, the handler parses the pipe-separated message and saves it to Supabase. This happens only once per user.

```
USER:   https://forms.gle/abc123

BOT:    Hi! I need your details once to fill forms for you.
        Send in this format:
        NAME | ROLL NUMBER | COLLEGE | BRANCH | YEAR

USER:   Rohit | 22EG1A0501 | Pragati Engineering College | CSE | 2nd
Year

BOT:    Profile saved! Now filling your form...
        [30 seconds later]
        Done! Score: 9 / 10
```

Every Subsequent Form

```
USER:   https://forms.gle/xyz789
BOT:    Got it Rohit! Filling your form...
        Done! Score: 10 / 10
```

Profile Update Command

```
USER:   UPDATE
        Rohit | 22EG1A0599 | Pragati | IT | 3rd Year
BOT:    Profile updated!
```

Personal Field Auto-Detection

profile_detector.py checks each question text against keyword lists. Personal fields are filled from the profile without involving Gemini AI at all — saving API calls and improving speed.

Question Contains Keyword	Data Source	Example Value
name, full name, student name	Profile: name	Rohit
roll, register number, regd	Profile: roll_number	22EG1A0501
college, institution, university	Profile: college	Pragati Engineering College
branch, department, stream	Profile: branch	CSE
year, semester, sem	Profile: year	2nd Year
email, mail	Profile: email	rohit@example.com
phone, mobile, contact	Profile: phone_number	919876543210
Everything else	Gemini AI	(AI generated answer)

---

10. Image Handling in Google Forms

Overview

Google Forms sometimes include images inside questions — diagrams, charts, circuit diagrams, or questions written entirely as images. AutoForm Bot handles all of these using Gemini Vision, which reads both text and images in a single API call.

Three Types of Image Situations

Type 1 — Image as Question Context

A diagram is shown and the question asks about it. Playwright takes a screenshot of the entire question block and sends it with the question text to Gemini Vision.

Type 2 — Images as Answer Options

The question text is normal but options are images. Playwright screenshots the full question block including all image options and sends it to Gemini Vision.

Type 3 — Decorative Images

College logos or banners at the top of the form. profile_detector.py detects and ignores these automatically — they are never sent to Gemini.

Gemini Vision Accuracy by Image Type

Image Type	Supported	Accuracy
Diagrams and charts	Yes	High (85-90%)
Circuit diagrams	Yes	High (85-90%)
Math equations as images	Yes	High (85-95%)
Graphs and plots	Yes	High (80-90%)
Maps and geography images	Yes	Medium (70-80%)
Images of shapes or symbols	Yes	Medium (70-80%)
Entire question written as image	Yes	Medium (65-75%)
Decorative logos and banners	Ignored automatically	N/A
Blurry or low quality images	Partial	Low (50-60%)

---

11. Development Timeline

Day	Task	Output
Day 1	Meta developer account + Gemini API key + Supabase project setup	All credentials ready
Day 2	FastAPI skeleton + /webhook GET verification + /webhook POST receiver	Webhook live and verified
Day 3	webhook_handler.py — full message routing logic in Python	Bot responds to all message types
Day 4	Playwright form scraper + filler working locally	Forms filled locally
Day 5	Gemini AI text + vision integration complete in ai_helper.py	AI answering questions
Day 6	profile_detector.py + database.py — full profile system working	User profiles stored and used
Day 7	whatsapp.py — Meta API replies integrated end to end	Full flow working locally
Day 8	Deploy to Render + environment variables configured	Live 24/7 in cloud
Day 9	End to end testing with real Google Forms	Production ready

---

12. Future Expansion

Phase 2 Features

- Form preview before submit — bot sends a summary of answers for the student to approve before submitting
- Bulk form filling — student sends multiple links at once, bot fills all simultaneously
- Retry on low score — if the form allows multiple attempts, bot retries until getting a better score
- Confirmation screenshot — Playwright screenshots the confirmation page and sends it as proof
- Attendance tracker — every attendance form submission is logged per user with percentage tracking
- Schedule submission — student specifies a time, bot submits at that exact time
- Deadline reminder — bot detects deadline text in message and sends a reminder before it
- Custom answers mode — student provides their own answers, bot just types them in automatically

Phase 3 — SaaS Product

- Free tier: 5 form submissions per month per user
- Premium tier (Rs. 99/month): unlimited submissions
- Usage tracking per phone number in Supabase
- Admin dashboard showing all users, submission counts, and revenue

This is a genuine startup idea. Every engineering college in India faces this exact problem. The total addressable market is millions of students.

---

13. End-to-End Flow Summary

```
Student forwards form link from WhatsApp group
        ↓
Meta Cloud API receives message
Fires POST request to FastAPI /webhook endpoint
        ↓
FastAPI immediately returns HTTP 200 to Meta
Runs processing in async background task
        ↓
webhook_handler.py takes over:
  → Extracts phone number + message text
  → Detects Google Form link
  → Queries Supabase for user profile
  → Sends 'Processing...' WhatsApp reply
```

```
        ↓
form_bot.py takes over:
  → Playwright opens form invisibly
  → Scrapes all questions + types
  → profile_detector separates personal vs quiz fields
  → Personal fields filled from Supabase profile
  → Quiz questions sent to Gemini AI (text or vision)
  → Gemini returns JSON answers
  → Playwright fills all fields + clicks Submit
  → Reads score from confirmation page
        ↓
webhook_handler.py receives score
whatsapp.py sends reply via Meta API:
  'Done! Your score: 9 / 10'
        ↓
Student receives WhatsApp message with score
Total time: 20 to 30 seconds
```

— End of Document —

Team INNOVEX · Pragati Engineering College